

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA0504

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Analytical Problem Solving (High)

Assessment Tool Weightage: 40%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Given a set of relations and sample data, write SQL queries to retrieve specific information using joins and subqueries, and explain the logic used. Bloom's Level: BL3 – Apply	BL4 – Analyze
2	Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations. Bloom's Level: BL4 – Analyse	BL5 – Evaluate
3	Given a real-world scenario (e.g., banking or student system), design a relational schema and construct appropriate SQL queries for data retrieval and updates. Bloom's Level: BL6 – Create	BL6 – Create
4	Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution. Bloom's Level: BL4 – Analyze	BL6 – Create
5	Compare different SQL approaches (joins vs subqueries) for solving the same problem and evaluate their efficiency. Bloom's Level: BL5 – Evaluate	BL4 – Analyze
6	Design and implement views for a database system to provide restricted data access, and justify their use for security and abstraction. Bloom's Level: BL6 – Create	BL5 – Evaluate

7	Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation. Bloom's Level: BL4 – Analyze	BL4 – Analyze
8	Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate
9	Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem. Bloom's Level: BL6 – Create	BL6 – Create
10	Given multiple relations, analyze the query requirements and decide whether to use views, joins, or subqueries, justifying your choice with reasoning. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate

Code of Conduct:

I E.Jeffrey Samuels(192524412) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Jeffrey Samuels

RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	Clearly interprets problem, identifies all parameters and requirements accurately	Understand s problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	Selects most appropriate method/formula a with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	Clearly interprets results with units and engineering relevance	Basic interpretation n with minor omissions	Limited or unclear interpretation	No interpretation

Q1. SQL Queries Using Joins and Subqueries (BL3 - BL4)

(a) Query Using a JOIN

Problem: List every student's name together with the title of each course they are enrolled in and the grade obtained.

```
SELECT S.Name, C.CTitle, E.Grade
FROM   Student S
JOIN   Enrollment E ON S.StudentID = E.StudentID
JOIN   Course C      ON E.CourseID  = C.CourseID
ORDER BY S.Name;
```

Logic: The three tables are related through foreign keys. Student is joined to Enrollment on StudentID to attach each enrollment record to the correct student, and Enrollment is joined to Course on CourseID to replace the course code with its title. The JOIN evaluates matching rows first, so only students who actually have an enrollment row appear.

Expected Output:

Name	CTitle	Grade
Kamesh	DBMS	A
Kamesh	Operating Systems	B
Jai	Computer Networks	A
Krishna	DBMS	A
Krishna	Data Structures	B
Abishek	DBMS	B
Abishek	Computer Networks	A

(b) Query Using a Subquery

Problem: List the names of students who are enrolled in the 'DBMS' course.

```
SELECT Name
FROM   Student
WHERE  StudentID IN (SELECT StudentID
```

```
FROM Enrollment
WHERE CourseID = (SELECT CourseID
                  FROM Course
                  WHERE CTitle = 'DBMS'));
```

Logic: The innermost subquery resolves the course title 'DBMS' to its CourseID (C201). The middle subquery then returns the set of StudentIDs enrolled in that course. The outer query filters Student rows whose StudentID lies in that set. Subqueries are useful here because the filtering condition (a specific course) is naturally expressed as a nested lookup rather than a join, and no columns from Course or Enrollment need to appear in the final result.

Expected Output: Kamesh,Krishna,Abishek

Q2. Relational Algebra for Multi-Condition Queries (BL4 - BL5)

Problem: Retrieve the names of students who scored grade 'A' in the 'DBMS' course, using relational algebra.

Relational algebra expression (built up in stages):

```
R1 = Student |X| Enrollment      (natural join on StudentID)
R2 = R1 |X| Course               (natural join on CourseID)
R3 = sigma(CTitle = 'DBMS' AND Grade = 'A') (R2)  (select rows matching both
conditions)
Result = pi(Name) (R3)           (project only the Name
column)
```

The full expression in one line is: $\pi(\text{Name}) (\sigma(\text{CTitle} = \text{'DBMS'} \text{ AND } \text{Grade} = \text{'A'}) (\text{Student} \bowtie \text{Enrollment} \bowtie \text{Course}))$.

Logic: The two joins first combine data from all three relations into a single wide relation. The selection (sigma) then applies both conditions simultaneously - the course must be 'DBMS' AND the grade must be 'A' - which is exactly how relational algebra expresses multiple conditions across relations: join first to bring the attributes together, then filter, then project down to only the attribute the question asks for.

Expected Output: Arjun, Kiran (Rahul is excluded because his DBMS grade is 'B', not 'A').

Q3. Schema Design and SQL for a Real-World Scenario (BL6)

Scenario: A bank needs to track customers, their accounts, and the transactions made on those accounts.

Relational Schema

Customer (CustomerID [PK], CName, Phone)
Account (AccountNo [PK], CustomerID [FK -> Customer], AccType, Balance)
Transaction(TransID [PK], AccountNo [FK -> Account], TransType, Amount, TransDate)

CustomerID is the primary key of Customer and is referenced as a foreign key in Account, establishing a one-to-many relationship (one customer can hold several accounts). AccountNo is similarly referenced from Transaction, so every transaction is tied to exactly one account, and transitively to one customer.

Sample Data

CustomerID	CName	Phone
Cust1	Ramesh	9876543210
Cust2	Sita	9876500000

AccountNo	CustomerID	AccType	Balance
A1001	Cust1	Savings	25000
A1002	Cust1	Current	120000
A1003	Cust2	Savings	54000

(a) Retrieval Query

Problem: List the names of customers together with the balance of their Savings accounts.

```
SELECT C.CName, A.Balance
FROM   Customer C
JOIN   Account A ON C.CustomerID = A.CustomerID
WHERE  A.AccType = 'Savings';
```

Expected Output:

CName	Balance
Ramesh	25000
Sita	54000

(b) Update Query

Problem: Ramesh deposits Rs. 5000 into his Current account (A1002); update the balance and record the transaction.

```
UPDATE Account
SET    Balance = Balance + 5000
WHERE  AccountNo = 'A1002';

INSERT INTO Transaction (TransID, AccountNo, TransType, Amount, TransDate)
VALUES ('T101', 'A1002', 'Deposit', 5000, '2026-08-01');
```

Logic: The UPDATE statement increments the existing balance rather than overwriting it, which keeps the change consistent regardless of the account's prior value. The INSERT then logs the deposit as a permanent transaction record, so the Transaction table maintains a full audit trail independent of the current Account balance.

Q4. Analysing a Nested Subquery and Predicting Its Output (BL4 - BL6)

Scenario schema: Department(DeptID, DeptName) and Employee(EmpID, Name, DeptID, Salary).

DeptID	DeptName
D1	IT
D2	HR
D3	Finance

EmpID	Name	DeptID	Salary
E1	Anand	D1	55000
E2	Bala	D1	62000
E3	Chitra	D2	48000
E4	Deepak	D2	51000
E5	Esha	D3	70000

EmpID	Name	DeptID	Salary
E6	Farook	D3	65000

Given Query:

```
SELECT Name, Salary
FROM   Employee E1
WHERE  Salary > (SELECT AVG(Salary)
                  FROM   Employee E2
                  WHERE  E2.DeptID = E1.DeptID);
```

Step-by-Step Execution

This is a correlated subquery: the inner query cannot be evaluated once for the whole table because it depends on E1.DeptID, which changes for every row of the outer query.

1. For each candidate row in E1, the inner query re-runs, restricted to rows of E2 sharing the same DeptID, and computes that department's average salary.
2. Department D1 (Anand 55000, Bala 62000): average = 58500. Anand (55000) fails the condition; Bala (62000) passes.
3. Department D2 (Chitra 48000, Deepak 51000): average = 49500. Chitra fails; Deepak (51000) passes.
4. Department D3 (Esha 70000, Farook 65000): average = 67500. Farook fails; Esha (70000) passes.
5. The outer query keeps only the rows that passed the comparison in their own department, and returns their Name and Salary.

Expected Output:

Name	Salary
Bala	62000
Deepak	51000
Esha	70000

Q5. Comparing Joins vs Subqueries (BL4 - BL5)

Problem: Solve 'list students enrolled in the DBMS course' using both a JOIN and a subquery (as in Q1), and evaluate which approach is more efficient.

Join Version

```
SELECT DISTINCT S.Name
FROM   Student S
JOIN   Enrollment E ON S.StudentID = E.StudentID
JOIN   Course C      ON E.CourseID  = C.CourseID
WHERE  C.CTitle = 'DBMS';
```

Subquery Version

```
SELECT Name
FROM   Student
WHERE  StudentID IN (SELECT StudentID FROM Enrollment
                     WHERE CourseID = (SELECT CourseID FROM Course
                                       WHERE CTitle = 'DBMS'));
```

Efficiency Evaluation

Both queries return the same three names (Arjun, Kiran, Rahul), but they are not always equivalent in cost. Most modern query optimizers rewrite a non-correlated IN subquery like this one into a semi-join internally, so on a well-indexed database the two forms typically perform similarly for this specific case. The difference becomes significant when the subquery is correlated (re-executed once per outer row, as in Q4) - that form scales poorly without an index on the correlating column, whereas an explicit JOIN lets the optimizer choose a single efficient strategy (hash join, merge join, or index nested-loop) over the whole relation at once. As a general rule: JOINS are preferable when columns from multiple tables must appear in the output or when the optimizer can exploit indexes across the join keys; subqueries are preferable for readability when the inner result is a simple filter (like resolving one course name to one ID) and no extra columns from the inner tables are needed.

Q6. Designing and Implementing Views (BL5 - BL6)

Scenario: General administrative staff should be able to see which students are taking which courses, but grades are sensitive and should be visible only to faculty. A VIEW is used to restrict the columns exposed to staff.

View Definition:

```
CREATE VIEW StudentCourseView AS
SELECT S.Name, C.CTitle
FROM   Student S
JOIN   Enrollment E ON S.StudentID = E.StudentID
JOIN   Course C     ON E.CourseID  = C.CourseID;
```

Usage by staff:

```
SELECT * FROM StudentCourseView WHERE CTitle = 'DBMS';
```

Justification

Security: the view exposes only Name and CTitle, permanently hiding the Grade column (and the internal StudentID/CourseID keys) from any role granted access to the view instead of the base tables - this is column-level access control enforced by the database itself, not by application logic.

Abstraction and logical data independence: staff-facing reports can query StudentCourseView with a single simple SELECT instead of repeating the three-table join every time, and if the underlying schema changes (e.g. a table is renamed or split), only the view definition needs to be updated - every dependent query keeps working unchanged.

Q7. Relational Algebra to SQL Conversion (BL4)

Problem: Find the titles of courses in which the student named 'Kiran' is enrolled.

Relational Algebra:

```
pi(CTitle) ( sigma(Name = 'Kiran') (Student |X| Enrollment |X| Course) )
```

Analysis of Differences in Representation

Relational algebra is procedural: the expression above literally dictates an execution order - join, then join again, then select, then project - and that order is part of the notation itself. SQL is declarative: it states what result is wanted (the CTitle where the name matches) without specifying the order of operations; the query optimizer is free to reorder the joins and selection (e.g. filter Student down to Kiran before joining, which is usually cheaper) as long as the final result is the same. Relational algebra also has no built-in way to name or alias its output beyond the projected attribute names, while SQL requires explicit table aliases (S, E, C) and lets columns be renamed with AS. Finally, relational algebra is a mathematical formalism used for reasoning about correctness and equivalence, whereas SQL is an executable language that a real DBMS parses, optimizes, and runs directly.

Expected Output: DBMS, Data Structures

Q8. Optimizing an Inefficient SQL Query (BL5)

Problem: For each student, count how many courses they are enrolled in.

Inefficient Version (Correlated Subquery in SELECT)

```
SELECT S.Name,  
       (SELECT COUNT(*) FROM Enrollment E  
        WHERE E.StudentID = S.StudentID) AS CourseCount  
FROM   Student S;
```

Why it is inefficient: the inner COUNT(*) subquery is re-executed once for every row returned by the outer query (once per student). Without an index on Enrollment.StudentID, each execution requires a full scan of Enrollment, giving roughly $O(n * m)$ work for n students and m enrollment rows - this scales poorly as either table grows.

Optimized Version (JOIN + GROUP BY, with an index)

```
CREATE INDEX idx_enrollment_studentid ON Enrollment(StudentID);  
  
SELECT S.Name, COUNT(E.CourseID) AS CourseCount  
FROM   Student S  
LEFT JOIN Enrollment E ON S.StudentID = E.StudentID  
GROUP BY S.Name;
```

Improvement: the LEFT JOIN (kept as LEFT so students with zero enrollments still appear, with CourseCount = 0) lets the optimizer scan each table once and match rows using the StudentID index, and GROUP BY then aggregates the counts in a single pass. This turns the earlier per-row re-scan into one index-assisted join followed by one aggregation - close to $O(n + m)$ instead of $O(n * m)$ - which is the standard trade-off: restructuring a correlated subquery into a join, backed by an index on the join column, removes the repeated inner-query cost.

Q9. Complex Query with Grouping, Aggregation, and a Nested Query (BL6)

Problem: List the titles of courses that have more students enrolled than the average number of students enrolled per course, along with their enrollment count.

```
SELECT C.CTitle, COUNT(E.StudentID) AS EnrollCount
```

```

FROM   Course C
JOIN   Enrollment E ON C.CourseID = E.CourseID
GROUP BY C.CTitle
HAVING COUNT(E.StudentID) > (
    SELECT AVG(CourseCount)
    FROM (
        SELECT COUNT(StudentID) AS CourseCount
        FROM   Enrollment
        GROUP BY CourseID
    ) AS CourseCounts
);

```

Logic: the innermost query groups Enrollment by CourseID and counts rows per course (DBMS = 3, Operating Systems = 1, Computer Networks = 2, Data Structures = 1). The middle subquery averages those four counts: $(3+1+2+1)/4 = 1.75$. The outer query then groups Course/Enrollment by course title, counts enrollments again for the final result set, and keeps only the courses whose count exceeds that average via HAVING - combining grouping, an aggregate function, and a nested query in one statement.

Expected Output:

CTitle	EnrollCount
DBMS	3
Computer Networks	2

Q10. Choosing Between Views, Joins, and Subqueries (BL5)

Scenario: A college wants (i) a permanent, reusable way for staff to see enrollment records without grades, (ii) a one-off report combining student, course, and grade data for the registrar, and (iii) a list of students enrolled in more than one course.

(i) Recurring restricted access -> VIEW

A VIEW is the right choice because the same column-restricted, joined result (Name + CTitle, no Grade) will be queried repeatedly by many staff members over time. Defining it once as StudentCourseView (see Q6) centralizes

both the security restriction and the join logic, so it does not need to be rewritten or re-explained for every future query.

(ii) One-off multi-table report -> JOIN

A direct JOIN across Student, Enrollment, and Course (as in Q1a) is preferable here because the registrar needs columns from all three relations simultaneously, and the query is not intended to be reused as a standing abstraction. Writing an explicit JOIN also gives the optimizer full freedom to choose the most efficient join order and strategy for a single execution.

(iii) Filtering by a derived count -> SUBQUERY

```
SELECT Name
FROM   Student
WHERE  StudentID IN (
    SELECT StudentID FROM Enrollment
    GROUP BY StudentID
    HAVING COUNT(*) > 1
);
```

A subquery is preferable here because the condition itself ('enrolled in more than one course') is a derived, aggregated fact about Enrollment alone - it does not need any columns from Student until after the qualifying StudentIDs are known. Expressing it as a subquery keeps the aggregation (GROUP BY / HAVING) isolated from the outer filter, which is clearer than folding everything into one combined JOIN/GROUP BY over all three tables when only Student.Name is ultimately needed.

General reasoning: choose a VIEW when the same restricted or combined shape of data will be reused and/or needs a standing security boundary; choose a JOIN when columns from multiple relations must appear together in a single, mostly one-time query; choose a subquery when a condition is most naturally expressed as 'a value/set computed from another relation', especially when that computation involves its own aggregation.